



# SAP BTP Pro-Code

Cloud Application Programming (CAP)

Version: 1.2

Date: 2026-01-14

## Overview and Important Notes

|                                           |          |
|-------------------------------------------|----------|
| <b>SAP BTP Pro-Code.....</b>              | <b>1</b> |
| <b>Overview and Important Notes .....</b> | <b>2</b> |
| <b>1. Setup.....</b>                      | <b>3</b> |
| <b>2. Task description.....</b>           | <b>3</b> |
| Helpful links .....                       | 3        |
| <b>3. Data Model .....</b>                | <b>4</b> |
| <b>4. Getting started.....</b>            | <b>5</b> |

## 1. Setup

1. Follow the [Getting Started Page](#) of the Capire Doc
2. Install the following extensions in VS Code:
  - [Cloud Foundry Tools](#)
  - [SAP Fiori Tools - Extension Pack](#)
  - [SAP CDS Language Support](#)
  - [Rainbow CSV](#)

## 2. Task description

Develop a comprehensive full-stack university management application using SAP Cloud Application Programming Model (CAP) that serves as a centralized system for managing academic operations. The application will handle the organization of academic programs (studies), course modules, and student enrollment in a structured and efficient manner.

This enterprise-grade solution will provide:

- **Study Program Management:** Organize and maintain different study programs (degrees) with their associated requirements including credits needed for completion, program duration, and curriculum structure
- **Module Catalog Management:** Define and manage course modules offered across study programs, including credit values, weights, validity periods, mandatory/elective status, exam options, and assignment tracking
- **Student Management:** Track students throughout their academic journey, managing their enrollment in study programs and registration for specific modules, maintaining comprehensive student records including personal information and academic progress
- **Module Assignment Tracking:** Enable students to register for available modules and manage their course selections, while enforcing business rules around module availability, prerequisites, and study program requirements
- **No Worries:** We won't implement all of this ;)

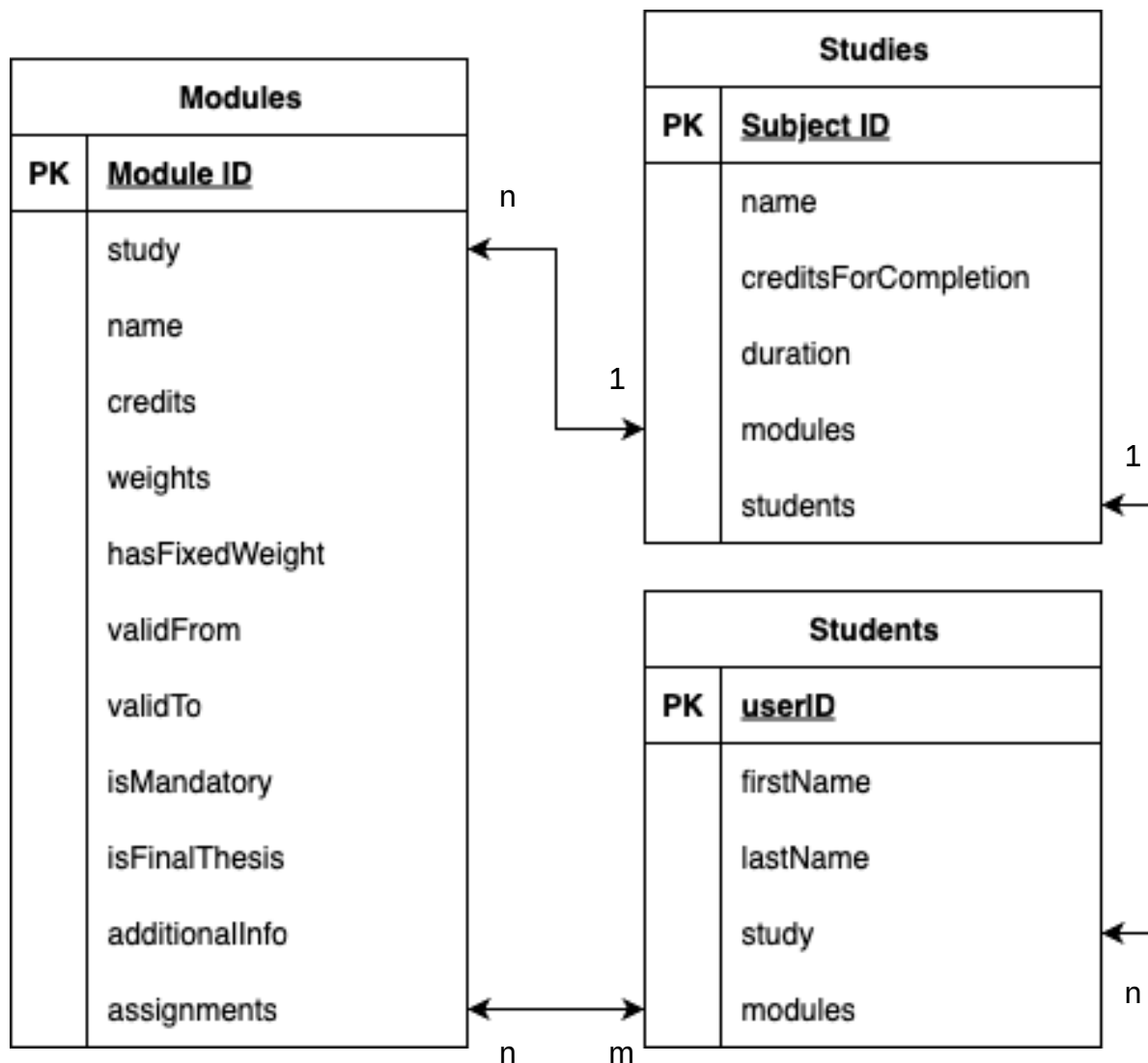
The system establishes clear relationships between Students, Studies (degree programs), and Modules, allowing for flexible curriculum design while maintaining data integrity and providing students with a personalized view of their academic path.

### Helpful links

- [Fiori development portal](#) & [Fiori elements documentation](#)
- [Internal CAPire \(CAP Documentation\)](#)
- [SAP Learning](#)

### 3. Data Model

Use belows diagram as a reference to design your CAP entities.

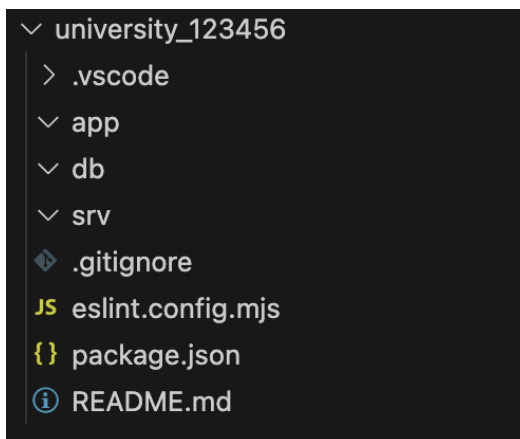


## 4. Implementation

1. Initialize your CAP application with the following command in the terminal either in VS Code or BAS

```
cds init university
cd university
```

2. Your project structure should look like this:



3. File structure setup:

Create database schema files in `/db/`:

- students.cds
- studies.cds

Create service definition files in `/srv/`:

- students-service.cds
- studies-service.cds
- catalog-service.cds

4. Create the data models based on the [entity diagram](#) in the `/db/` folder. You can have a look at the [CDL documentation for reference](#)

**Hint:** if you are stuck with the creation of the data model, please raise your hand. We are happy to help you. 🙋

5. Create example data by running the following command:

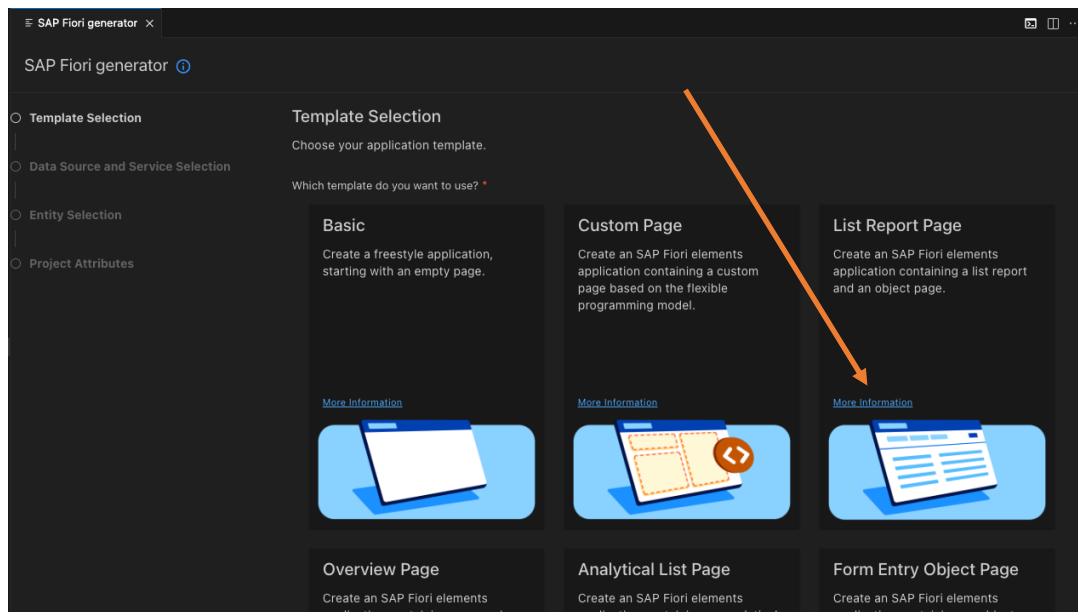
```
# Generate schema with example data  
cds add data --records 10
```

6. Test if your data models are defined correctly and no compilation errors occur  
(You can leave this command running while working on the next steps):

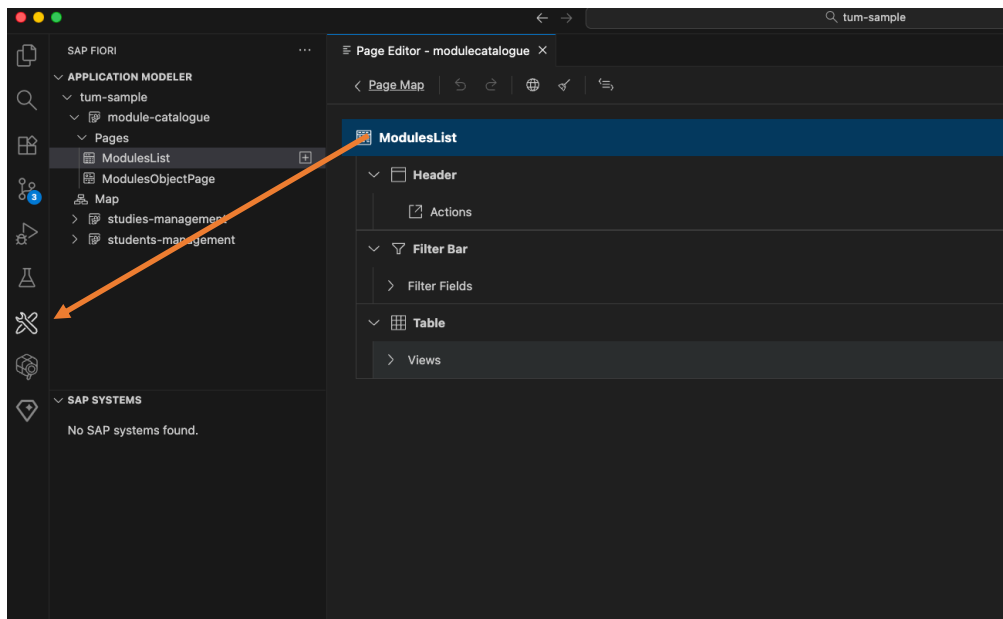
```
cds watch
```

7. Create the services in the `/srv/` folder, which expose the entities via API.

- Check out the documentation for [how to define services](#).
- The studies service should expose the **Studies** entity.
- The students service should expose the **Students** entity and the **Studies** entity. The studies entity can be later used as a value help when creating a new student and assigning them to a study.
- In the Students and Studies service [enable draft](#) for the respective entities to allow them being edited.
- In the Catalog service expose the **Modules** entity as well as the **ModuleAssignments** entity, to show a list of modules students can sign up for.
- 8. Actions allow to define POST endpoints, which execute some logic when being called. Follow the [docs](#) to add a bound action “assign” to the **Modules**. Do not add an implementation for that yet. We will do that later.
- 9. Create UI applications by using the Fiori Frontend Generator.  
**(Note: You should repeat this step for each OData service, in total 3 times)**
  - In the menu click **View->Command Palette (Ctrl + Shift + P)** and start typing Fiori and select **Fiori: Open CF Application Generator**
  - Select **List Report Page** in the template section and click “Next”



- In *Data Source* select **“Use a Local CAP Project”**:
  - Choose a CAP Project: **university**
  - OData Service: **YourService**
  - In the *Entity Selection* select:
    - Main Entity: **YourMainEntityOfTheService**
    - Navigation Entity: **YourNavigationEntity** (If applicable)
    - Automatically add table columns to the list page and a section to the object page if none already exists: **Yes**
    - Table Type: **Responsive**
  - In the *Project Attributes* select:
    - Module Name: **e.g. students-management** for the students-service
    - Application Title: **e.g. Students Management** for the students service
    - Application Namespace: **empty**
    - Description: **YourDescription**
    - Enable TypeScript: **No**
    - Add Fiori Launchpad Configuration: **Yes**
  - In the SAP Fiori Launchpad Configuration:
    - Semantic Object: **YourMainEntityOfTheService**
    - Action: **manage** (or something better fitting)
    - Title: Choose accordingly, will be displayed on the Launchpad tile
    - Subtitle: Choose accordingly, will be displayed on the Launchpad tile
  - Continue creating the Fiori config for the remaining services, once you finished, continue with the next step.
10. For defining the UI annotations, you can use the Fiori application modeler plugin (🔗 icon):



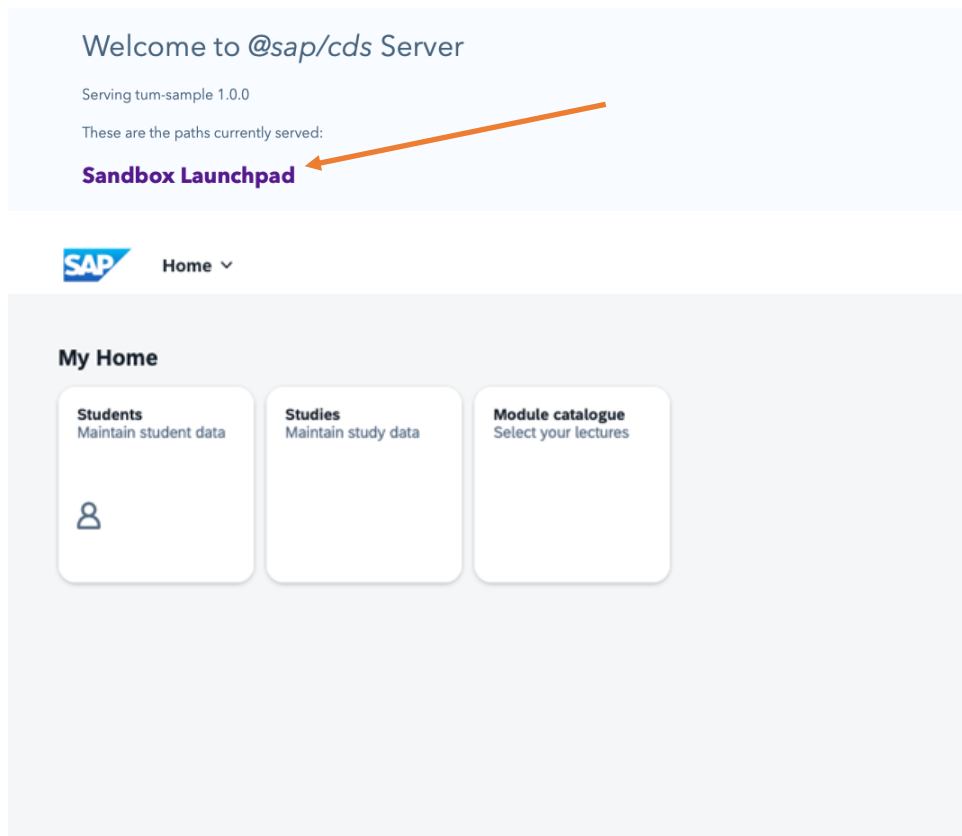
- Select the app you want to work on and select the page, e.g. either the list page or the details page.
- Afterwards you can configure columns for tables or modify the sections shown on the details page.
- FYI, but not necessary: You can also manually define the annotations in cds files. Ideally use the *annotations.cds* file in each app folder to do the annotations for the respective app.
  - i. **@UI.LineItem** specifies a table definition
  - ii. **@UI.HeaderInfo** specifies the header of an object page as well as the table name
  - iii. **@UI.PresentationVariant** specifies the default sorting for a table
  - iv. **@UI.FieldGroup** specifies a form
  - v. **@UI.Facets** specifies the sections of an object page. Facets can contain field groups, line items or charts.
  - vi. You can check-out the [Fiori elements showcase](#) for samples of each annotation.
- To disable the “Go” button in the UI add the “liveMode” setting in each applications webapp/`manifest.json` file. For more information, [reference to the docs](#).

11. Add a local launchpad by running the following command:

```
npm add -D cds-launchpad-plugin
```

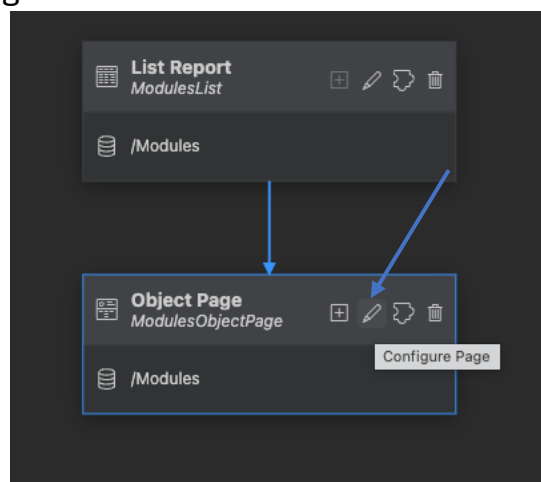
Afterwards usually on localhost, port 4004 a “Sandbox Launchpad” appears, via which you can test the UI apps within the [SAP Shell](#).





## 12. Add the actions to the UI

- After the creation of the basic UI application, add Buttons to call the assign and unassign actions you created earlier. Again, use the Fiori Tools to open the Page Map of the catalog application or execute the Command Fiori: Show Page Map and select the catalog app.
- Edit the Object Page:



- Under Header => Actions => + on the right side => Add Actions => Select the assign action
- If you now restart, go to the object page and click assign, you should get an error that an implementation is missing. We will add that later.

## 13. Debug your application

- You can start your application in debug mode by using a *JavaScript Debug Terminal* in VSCode and executing `cds watch` there

#### 14. Adding authorization and authentication to your application

- You now want to restrict the catalog service so that only Students can see it. Read about auth in CAP [here](#).
- CAP offers pre-defined mock users. We now want to overwrite one of them to add the role *Student* to the mock user alice. You can do it like this in your *package.json*:

```
"cds": {
  "requires": {
    "[development]": {
      "auth": {
        "kind": "mocked",
        "users": {
          "alice": {
            "password": "1234",
            "roles": [
              "Student"
            ]
          }
        }
      }
    }
  }
}
```

•

#### 15. Adding business logic to your application

- Run the following commands to add the TypeScript configuration:

```
cds add typescript
npm install
```

- Add a `/srv/catalog-service.ts` file. CAP will automatically consider js/ts files with the same name as the service file.

```
import cds from '@sap/cds';
import { Module } from '#cds-models/CatalogService'

export class CatalogService extends cds.ApplicationService {
  async init() {

    this.on(Module.actions.assign, async req => {
      console.log("Assign called", req.user)
    })

    return super.init();
  }
}
```

- Implement the assign action that assigns the user to the Module. In case an assignment already exists, do not add one but return an error.
  - i. You can leverage [req.subject](#) as a shortcut for the information which entity is targeted by the bound action.
  - ii. You can use [req.user](#) to gain information about the logged in user
  - iii. [req.params](#) contains the URL parameters like the ID of the bound entity.
  - iv. Use CAPs SQL like [query API](#) to do the necessary statements.
  - v. Use [req.info](#) to let the student know that you assigned them or [req.error](#) to display the error.
  - vi. Make use of the debugger to inspect the incoming requests or results from database access.

Congratulations, you just created your first CAP application!!!